Assignment 3 NLP — краткий разбор для защиты

1. Что реализовано и где это искать

Сжатая версия: методы, существенные решения в коде, реальные результаты и ограничения. Без основ Python, списка импортов и полного повторения кода. Номера блоков ниже считают **только кодовые ячейки**: их 21. Ячеек вместе с Markdown — 39.

Задача	Данные и требуемый результат	Наш метод
Тональность	Отзывы; negative / neutral / positive	VADER и TF-IDF + Logistic Regression
Суммаризация	Первые 3 статьи: по 3 предложения и общий top-6; все 5: новая сводка	TF-IDF-центроид и предобученная T5-small
Песни	4 жанра × 10 текстов; частоты, категории, редкие слова	Лемматизация, Counter, словари категорий
Язык	6 языков, по 800 примеров; сравнение методов	8 сочетаний очистки/TF-IDF/модели и fastText

Источники: отзывы — Twitter US Airline Sentiment, публичная копия `martinpella/twitter-airlines`; песни — `etarakci/music-genre-prediction` (AZLyrics + жанры Spotify); язык — WiLI-2018, зеркало `alianoroozi/Language-Identification`. Пять статей приложены к заданию и встроены в ноутбук. URL загрузок закреплены Git-коммитами; Kaggle-токен и API-ключи не нужны.

Главное различие: LR и NB обучаются на наших размеченных данных; VADER использует готовый словарь; T5 и fastText используют готовые веса. В задаче песен модели не обучаются.

Блоки	Назначение
1–3	Окружение, общие функции, английская предобработка
4–8	Отзывы → split → VADER/LR → оценка → ошибки и визуализация
9–13	Статьи → извлечение → T5 → оценка сводок
14–15	Выбор песен, top-10, категории, два вида редкости
16–20	WiLI → 8 моделей → fastText → ошибки → короткие/длинные входы
21	Версии, источники, контрольные суммы и размеры выборок

Как пользоваться: читать этот PDF вместе с сохранёнными результатами ноутбука. Не нужно пересказывать каждую техническую строку: на защите важнее объяснить, почему выбраны данные и методы, как проверялась независимость test и что означают ошибки.

2. Тональность: подготовка данных и честная проверка

Блоки 3–4. Выбраны отзывы об авиакомпаниях: у них уже есть три размеченных класса. Обычный IMDb чаще бинарный; без новой разметки нейтральный класс на нём честно не оценить.

Предобработка для ML

`english_tokens()` выполняет: HTML-декодирование → lowercase → раскрытие части сокращений → удаление URL и аккаунтов → выделение английских слов → POS-теги → стоп-слова → леммы. `sentiment_preprocess()` соединяет леммы в строку для TF-IDF; кеширование ускоряет повторы.

Почему сохраняем отрицания: удаление `not` превращает `not good` в `good`. `STOP_EN - NEGATIONS` исключает отрицания из удаляемых слов. Сокращения вроде `can't` раскрываются, но это набор правил, не полное понимание грамматики.

Зачем POS при лемматизации: WordNet по умолчанию предполагает существительное; глагол `running` нужно обрабатывать как глагол, чтобы получить `run`. Лемматизация уменьшает число словоформ; stemming обычно лишь обрезает окончания.

VADER получает **исходный text**, ML — **processed**. Для VADER полезны регистр, пунктуация, усилители и отрицания; после общей очистки часть этих сигналов пропала бы. Поэтому сравниваются два практически настроенных подхода с разной предобработкой.

Дубликаты и split

После очистки исключаются пустые processed, одинаковые processed с конфликтующими метками, затем оставшиеся повторы. Только после этого выполняется стратифицированный split.

Класс	Train	Validation	Test
Negative	5 409	1 803	1 804
Neutral	1 703	567	568
Positive	1 276	426	425
Всего	8 388	2 796	2 797

`train_test_split(test_size=.4)` оставляет 60% train; второй split делит остаток пополам. `stratify=label` сохраняет близкие доли классов. `isdisjoint()` проверяет, что одинаковые processed не пересекаются между наборами. `SEED=42` фиксирует случайное разбиение.

Защита от утечки: TF-IDF обучается после split только на обучающей части; пороги и C выбираются на validation. Детерминированная очистка заранее допустима, но исключение неоднозначных примеров меняет совокупность: метрики относятся к очищенным отзывам. Отдельная проблема для реального применения — перенос на другой домен и другую эпоху.

3. Тональность: VADER и TF-IDF + LR

Блок 5 — словарный подход. `SentimentIntensityAnalyzer().polarity_scores(text)` возвращает оценки VADER. Используем `compound` $\in [-1,1]$: это эмоциональная оценка, **не вероятность положительного класса**.

```
score < lower → negative
score > upper → positive
иначе → neutral
```

Пары lower/upper перебираются на validation; критерий выбора — macro F1. Получено **lower=0.0, upper=0.4**. Значит −0.3 → negative, 0.2 → neutral, 0.7 → positive. Ровно 0 и 0.4 относятся к neutral: в условиях строгие сравнения. Словарь VADER мы не обучали; настроили только пороги его решения.

Блок 6 — машинное обучение. `make_sentiment_model(C)` создаёт новый `Pipeline`: `TfidfVectorizer → LogisticRegression`. В `fit()` векторизатор изучает словарь/IDF, а LR — коэффициенты классов. `predict()` использует уже выученные преобразования.

Параметр	Назначение
ngram_range=(1,2)	Слова и соседние пары, например good и not good
min_df=2	Не учитывать признаки, встречающиеся меньше чем в 2 документах
max_df=.98	Не учитывать признаки более чем в 98% документов
max_features=35000	Ограничить размер словаря
sublinear_tf=True	Использовать 1+log(count) вместо сырой частоты
class_weight='balanced'	Увеличить относительный вклад ошибок редких классов
C	Обратная сила регуляризации: меньше C — сильнее ограничение весов

TF-IDF подчёркивает признаки, отличающие документы. Со сглаживанием в scikit-learn: `idf = log((1+N)/(1+df)) + 1`; затем по умолчанию L2-нормирование вектора. Матрица разреженная: у одного отзыва ненулевых признаков мало относительно всего словаря.

LR строит взвешенные суммы признаков по классам и преобразует их в вероятности. Регуляризация сдерживает слишком большие веса, уменьшая риск переобучения. Balanced не создаёт новые отзывы и не гарантирует одинаковый recall классов.

Перебираются C = 0.5, 1, 2; выбран **C=2** по validation macro F1. Затем новый pipeline обучается на train + validation, включая повторный fit TF-IDF. Test не участвует в выборе и остаётся для итогового сравнения. `max_iter=1200` — лимит итераций оптимизатора, а не число эпох нейросети. Cross-validation в этой работе не проводилась.

4. Тональность: результаты, ошибки и графики

Блоки 6–8. Итоговая оценка выполнена на 2 797 test-отзывах.

Метод	Accuracy	Macro F1
Majority: всегда negative	64.50%	0.2614
VADER, выбранные пороги	55.85%	0.5326
TF-IDF + LR	76.87%	0.7057

Ключевой вывод: VADER проигрывает majority по accuracy, но выигрывает по macro F1. Постоянный negative использует дисбаланс и не распознаёт другие классы. LR лучше обоих по двум метрикам. У неё F1 negative ≈0.86, neutral ≈0.57, positive ≈0.69: нейтральный класс остаётся сложным. Общая accuracy не означает одинаковое качество классов.

Что читать в метриках

- Accuracy — доля правильных ответов среди всех объектов.
- Precision класса — какая доля предсказаний этого класса верна: TP/(TP+FP).
- Recall — какая доля истинных объектов класса найдена: TP/(TP+FN).
- F1 — гармоническое среднее precision и recall.
- Macro F1 — простое среднее F1 классов; редкий класс весит столько же, сколько частый.
- Weighted F1 учитывает размер классов. В обычной однозначной многоклассовой задаче micro F1 совпадает с accuracy.

`classification_report()` показывает показатели по классам; support — число истинных примеров. В confusion matrix строки — истинные классы, столбцы — предсказанные. Диагональ — правильные ответы; остальные клетки показывают направления ошибок.

Что показал анализ ошибок

`assign(predicted=...).query('label != predicted')` выделяет неверные ответы. Первые 8 ошибок каждого метода выводятся с гипотезами причин. Это иллюстрации, а не случайная репрезентативная выборка всех ошибок. Проверка наличия good/thanks или отрицания не доказывает сарказм автоматически.

В примере **“Great, another three-hour delay.”** VADER дал positive, ML — negative. Great положительное по словарю, но в жалобе на задержку звучит иронично. Один удачный ответ ML не доказывает, что она понимает любой сарказм.

Графики: WordCloud показывает частоты слов в train+validation, а не качество модели. `coef_` LR показывает связь признаков с линейной оценкой класса. Частое слово может плохо различать классы, а большой коэффициент не доказывает причинность. Улучшения: область действия отрицания, доменная лексика, проверка разметки, контекстные модели и отдельный набор ироничных сообщений.

5. Суммаризация: извлечение предложений

Блоки 9–10. Из первых трёх статей требуются по 3 ключевых предложения и общий список из 6. Это **extractive summarization**: выбранные предложения копируются, новый текст не пишется.

Статьи посвящены музыкальному плагиату: извлечению/сравнению мелодии, индексированию и аудиоотпечаткам. Их содержание — данные для NLP; мы не реализуем аудиосистему и не вычисляем MFCC в этом ноутбуке.

Алгоритм

1. `clean_article()` убирает управляющие символы, переносы слов, некоторые заголовки, ссылки вида [12] и лишние пробелы. Исходники встроены; локальный файл при наличии их заменяет.
2. `nltk.sent_tokenize()` делит первые 3 документа на предложения. Исключаются фрагменты короче 5 слов и повторы.
3. `sentence_key` игнорирует регистр и пунктуацию; по нему дополнительно удаляются повторы внутри документа.
4. Один `TfidfVectorizer` строит **общее пространство** предложений трёх статей.
5. `sentence_X.mean(axis=0)` даёт центроид — средний вектор общей темы.
6. `cosine_similarity(sentence_X, centroid)` становится score предложения.
7. В каждой статье выбираются 3 максимальных score и восстанавливается исходный порядок. Для общего списка сортируется весь корпус, убираются одинаковые sentence_key и берётся top-6.

```
Предложения → общий TF-IDF → центроид → сходство с центроидом → ранжирование
```

Косинусное сходство: `a·b / (||a|| × ||b||)`. Оно сравнивает направление векторов, снижая влияние различий длины. Здесь используются неотрицательные TF-IDF-признаки. Score показывает тематическую типичность, **не гарантированную научную значимость**.

Почему результаты ограничены

Общая музыкальная лексика может поднять фразу вроде «раздел 5 описывает эксперименты». Точное нормализованное совпадение удаляется, но смысловые повторы остаются. В top-6 нет квот по статьям: он может сильнее представлять одну из них. Общий список отсортирован по весу; это может ухудшить связность чтения.

На защите результат точнее назвать **шестью наиболее весомыми уникальными нормализованными предложениями**, а не абсолютно лучшей сводкой. Предложения улучшения: фильтровать описания структуры статьи, использовать MMR для разнообразия, тематические квоты и восстановление порядка. TextRank — другой возможный метод на графе сходства; в текущем коде его нет.

6. Суммаризация: T5 и обработка пяти документов

Блоки 11–12. Используется готовая **T5-small**, экспортированная в квантованный INT8 ONNX. `AutoTokenizer` преобразует текст в токены; `ORTModelForSeq2SeqLM` выполняет модель через ONNX Runtime на CPU. `torch` используется для входных тензоров и режима inference. Мы не обучаем и не дообучаем T5.

T5 — encoder–decoder: encoder представляет вход, decoder генерирует выход, attention связывает его с входным контекстом. ONNX — формат выполнения, не другая архитектура. INT8 экономит память, но может менять качество; отдельно эффект квантования не измерялся.

Почему нужна иерархия

Ограничение входа — 512 токенов. Токен не равен слову: слово может делиться на части. Обрезать весь корпус до начала означало бы потерять последние статьи.

```
Каждая статья → части по 430 токенов → сводки частей
→ объединение сводок → при необходимости повторное сжатие → общий текст
```

`token_chunks()` нарезает IDs. `summarize_chunk()` добавляет префикс `summarize:`, проверяет реальную длину входа и вызывает `generate()`. Запас до 512 нужен для префикса, служебных токенов и повторного кодирования. Резка по токенам может оборвать предложение.

Обработаны все 5 статей: 2+3+2+2+1 = **10 исходных блоков**. Если объединённые сводки длиннее 430 токенов, цикл сжимает их снова, максимум 8 уровней. Проверяется, что длина уменьшается. Прохождение всех статей через модель не гарантирует их полного отражения в финале.

Параметр генерации	Смысл
max_new_tokens	Верхняя граница новых токенов: 75 для исходных частей, до 150 в финале
min_new_tokens=20	Не завершать выход слишком рано
num_beams=4, do_sample=False	Beam search вместо случайного sampling
no_repeat_ngram_size=3	Запрет повторения выходных троек токенов
repetition_penalty=1.1	Дополнительный штраф повторений

Настоящий итог: 35 слов о переводе мелодии в ноты, сходстве с базой и происхождении Amen Break. Потеряны MFCC/энтропия/энергия, индексирование, сэмплирование; у слова break пропало уточнение Amen. Начала предложений со строчной буквы и переходы слабы. Это слишком неполная сводка. Отдельная вычитанная аннотация в Markdown — **авторская редактура**, не результат generate. На защите это разделение нужно сохранять.

7. Суммаризация: как оценивалось качество

Блок 13. Для исходников нет независимых готовых эталонов. В решении написаны две авторские учебные сводки: для 3 статей и для 5. Это ограниченная оценка, не экспертный benchmark.

`RougeScorer(['rouge1','rouge2','rougeL'], use_stemmer=True)` сравнивает кандидата с соответствующим эталоном. В `score(reference, summary)` сначала передаётся эталон.

- ROUGE-1 — совпадение отдельных слов; ROUGE-2 — соседних пар.
- ROUGE-L — общая подпоследовательность с сохранением порядка, не обязательно непрерывная.
- В таблице показаны F1 этих совпадений. Корректное перефразирование может получить низкий балл.
- ROUGE не проверяет грамматику, истинность или логическое следование.

Результат	Слов	Доля исходника	R-1 F1	R-2 F1	R-L F1
Extractive, 3 статьи	137	7.78%	0.3153	0.0364	0.1892
T5, 5 статей	35	1.35%	0.1818	0.0000	0.0979

`compression_ratio` = слова сводки / слова соответствующих источников. Первая тройка содержит 1 761 слово, все пять — 2 602. Меньшая доля не означает лучшее качество: у T5 сильное сжатие достигнуто ценой пропущенных тем. Нулевой ROUGE-2 не доказывает ложность всех фактов — не совпали обработанные биграммы.

Почему нельзя объявить победителя по этой таблице: у двух методов разные входы и разные эталоны. Для прямого сравнения нужны одинаковые источники и протокол оценки.

Дополнительные проверки

`task2_document_coverage.csv` содержит сходство сводки с каждым документом в TF-IDF. Это **лексический прокси покрытия**, а не проверка сохранения всех важных идей. `task2_factual_support_review.csv` сопоставляет каждому сгенерированному предложению наиболее похожее исходное. Это материал для ручной проверки, не доказательство факта: «метод работает» и «метод не работает» могут иметь близкую лексику.

Оценивать следует три свойства: **полноту** важных тем, **связность** переходов и ссылок, **корректность** фактов. У extractive нет новых утверждений, но вырывание из контекста может изменить смысл. У генерации возможны галлюцинации и смешивание деталей.

Содержательно пять статей охватывают мелодическое сравнение, быстрый поиск, аудиоотпечатки, сэмплирование и ритм. Эти темы отражены в авторской редактуре; нельзя приписывать её полноту автоматической T5. Следующий шаг — более сильная модель, лучшее покрытие документов и независимые эталоны с ручной проверкой фактов.

8. Песни: частоты, категории и редкость

Блоки 14–15. Из `genre_lyrics_data.csv` выбираются country, blues, folk, hip-hop, по 10 текстов с `random_state=42`. Перед выбором удаляются пропуски, короткие тексты (менее 30 слов) и повторы нормализованного текста. Assert проверяет 40 уникальных текстов и 10 на жанр. У CSV нет названий/исполнителей: сохраняются source_row и SHA-256.

Для каждого текста выполняется английская очистка и POS-лемматизация, здесь без отдельного сохранения отрицаний. `Counter` объединяет токены жанра; `most_common(10)` выдаёт top-10. Обучения классификатора в этой задаче нет.

Жанр	Самое частое слово	Редких слов: 1–3 вхождения	Body parts / 1000 токенов
Country	know: 52	372	12.36
Blues	oh: 73	245	17.63
Folk	get: 31	370	29.84
Hip-hop	im: 65	629	6.88

Источник уже частично очищен: `im`, `dont`, `cant` встречаются без апострофов. Наши правила для `can't` не восстановят `cant`. Это объясняет часть top-10 и показывает ограничение предобработки; нужно проверять словарь сокращений, а не скрывать результат.

Категории

`CATEGORIES` задаёт слова-леммы: время суток, сезоны, части тела. Например, eyes → eye и hands → hand. Для каждого слова считаются абсолютная частота и **1000 × частота / все обработанные токены жанра**. Затем частоты суммируются по категории. Нормирование нужно, потому что даже по 10 песен жанры могут иметь разный объём текста.

Во всех четырёх выборках body_parts чаще двух других заданных категорий. Это не означает, что большинство песен «о теле»: большая часть слов вне словарей. Heart бывает метафорой, fall — глаголом, spring — не сезоном; словарь не снимает многозначность.

Две трактовки редкости из PDF

1. Слова с частотой **1–3 в конкретном жанре**: полный список отдельно для каждого жанра.
2. Слова, присутствующие **во всех 4 жанрах** и в каждом с частотой **1–3**: пересечение словарей + проверка `all(...)`.

Во втором списке 15 слов. Например, morning: 1/3/1/3 по country/blues/folk/hip-hop; всего 8 вхождений, но условие относится к каждому жанру, не к сумме. Редкость не равна художественной значимости: редкими бывают имена и опечатки. Повторы припевов и маленькая выборка ограничивают обобщение на жанры в целом.

9. Определение языка: эксперимент и модели

Блоки 16–18. Выбраны English, Russian, French, German, Spanish, Portuguese. Испанский/португальский — близкая пара. После дедупликации и исключения конфликтных меток берём **800 объектов на язык**: 640 train + 160 test. Итого 4 800 объектов, split 80/20: **3 840 / 960**.

Оба исходных набора WiLI объединены и заново разделены. Это выполняет учебный протокол, но не воспроизводит официальный benchmark WiLI. Один объект часто является абзацем. `zip_longest()` проверяет совпадение количества строк x и y; canonical после очистки позволяет исключить одинаковые тексты между итоговыми наборами.

Восемь контролируемых сочетаний

Измерение	Варианты
Очистка	Minimal: Unicode NFKC + пробелы; aggressive: дополнительно lowercase и только буквы/пробелы
TF-IDF	Char n-граммы 1–3; word униграммы
Классификатор	MultinomialNB(alpha=.1); LR(C=3, max_iter=1000)

2 × 2 × 2 = **8 Pipeline**. Векторизатор учится только на train, min_df=2, max_features=60000, sublinear_tf=True. Настройки фиксированы заранее, validation здесь нет. Таблица test — сравнение, не основание бесконечно подбирать новые настройки по test.

Unicode: `isalpha()` сохраняет кириллицу и характерные буквы é, ñ, ə, қ. Шаблон только a–z разрушил бы важные сигналы. Стоп-слова и лемматизация не применяются: служебные слова и окончания сами помогают определять язык. Сохранность казахских букв не добавляет казахский как класс: его нет в наших обучающих метках.

Char-признаки могут работать на незнакомом слове по знакомым сочетаниям букв. При minimal они могут содержать также пробелы/пунктуацию. Word-признаки сильнее зависят от словаря; невиданные слова не добавляются при predict.

Naive Bayes: оценивает типичность признаков для класса при упрощённой условной независимости; alpha сглаживает нулевые оценки. Это допущение не буквально верно для соседних букв, но модель остаётся полезной. NB здесь получает неотрицательные TF-IDF-веса.

FastText lid.176 — девятый, уже обученный baseline на 176 языков. Файл .ftz компактный; загрузка с официального сервера либо зеркала проверяется одной ожидаемой SHA-256. Ответ вне шести языков не подменяется разрешённым классом и считается ошибкой. Модель обучалась на внешних данных, включая Wikipedia: возможно пересечение источников с WiLI, поэтому сравнение нельзя назвать полностью независимым.

10. Языки: результаты, ошибки и короткий ввод

Блоки 19–20. Основной метод заранее зафиксирован как **minimal/char/LR**.

Очистка / признаки / модель	Accuracy	Macro F1
Minimal / char / NB	98.02%	0.9805
Minimal / char / LR	99.06%	0.9907
Minimal / word / NB	99.06%	0.9907
Minimal / word / LR	98.85%	0.9886
Aggressive / char / NB	97.81%	0.9785
Aggressive / char / LR	98.85%	0.9886
Aggressive / word / NB	98.75%	0.9876
Aggressive / word / LR	98.96%	0.9897
FastText lid.176	98.12%	0.9820

Minimal лучше в 3 из 4 пар, но word/LR выигрывает от aggressive. Значимость различий не проверялась. Word/NB чуть выше основной LR по неокруглённому F1: char не всегда лучше.

Основная LR: **951/960 правильных**, 9 ошибок. Часто: Portuguese→English — 4, Russian→English — 2, French→English — 1. В примерах — английские библиографические записи: метка отражает язык страницы, но не всегда самого фрагмента. Это не только путаница близких языков.

Стресс-тест длины

Из test берём 3 первых слова, 1 или 2 предложения. Если второго нет, добавляем его из другого test-текста того же языка с пометкой. Совпадения с train исключаются.

Метод	3 слова, n=820	1 предложение, n=945	2 предложения, n=960
Minimal / char / LR	84.51%	98.52%	99.06%
Minimal / char / NB	90.24%	97.88%	98.23%
Minimal / word / LR	51.46%	96.19%	98.33%
FastText	88.17%	97.46%	98.12%

Ограничения: варианты связаны, размеры групп различаются; «1–3 words» фактически означает ровно 3 слова. Разбиение NLTK не настроено отдельно для каждого языка.

`identify_language()` выдаёт **некалиброванные** вероятности. LR всегда выбирает из шести языков, даже для неизвестного. 0.9 не гарантирует 90% верных ответов: нужна калибровка.

11. Контрольные вопросы: короткие ответы

1. Extractive и abstractive? Первый выбирает исходные предложения, второй пишет новые формулировки. Наши реализации — центроид и T5; генерация может искажать факты.

2. Почему TF-IDF/GraphRank выбирает предложения? В суммаризации единицей ранжирования назначено предложение. Отдельные слова дают признаки, но не готовую связную сводку. В нашем коде GraphRank/TextRank не реализован.

3. Главные трудности генерации? Сохранить важные темы, не галлюцинировать, связать документы и уложиться в контекст. В иерархии информация может теряться на каждом уровне.

4. Что такое ROUGE? Совпадения с эталоном: слова, биграммы и общая подпоследовательность. Измеряет лексическое сходство; не заменяет проверку истинности и качества парафраза.

5. Зачем грамматика в абстрактивной сводке? Ошибочная связь, отрицание или местоимение может изменить смысл, даже если отдельные ключевые слова сохранены.

6. Словарь и ML в тональности? VADER использует готовые оценки слов и правила; LR учит веса по размеченным отзывам. Пороги VADER всё же подбирались на validation.

7. Зачем лемматизация? Объединить словоформы и уменьшить разреженность. POS помогает выбрать правильную лемму; сленг и многозначность автоматически не решаются.

8. Что показывает confusion matrix? Какие истинные классы куда предсказаны. Диагональ — правильные ответы; вне диагонали — направления ошибок.

9. Почему accuracy бывает обманчива? Большинство может скрыть провал редких классов. Наш majority имеет 64.5% accuracy, но macro F1 лишь 0.2614.

10. Что мешает тональности? Отрицания, сарказм, смешанные эмоции, доменная лексика, сленг, недостаток контекста и шумная разметка.

11. Нормализация и стандартизация? Стандартизация чисел обычно `(x−mean)/std`; нормализация — масштаб/норма вектора. Нормализация текста — другое понятие: регистр, Unicode, формы слов. Здесь есть L2-норма TF-IDF, но нет StandardScaler.

12. TF-IDF против частот? Уменьшает роль повсеместных слов, выделяет характерные. Для буквального подсчёта упоминаний в песнях нужны именно частоты, не TF-IDF.

13. Почему генератор ошибается грамматически? Предсказывает вероятные продолжения, а не проверяет строгие правила; мешают шум, домен, длинные зависимости и малая модель.

14. ROC против PR? ROC: recall/TPR против FPR; PR: precision против recall. При редком positive PR часто лучше показывает цену ложных срабатываний. Наш ноутбук эти кривые не строит; для multiclass нужен явный протокол, например one-vs-rest.

15. Bag-of-Words? Вектор количеств слов из словаря, обычно без порядка. Биграммы добавляют локальный порядок; TF-IDF изменяет веса признаков.

12. Что ещё могут спросить на защите

Где обучаемые параметры, а где гиперпараметры? Веса LR учатся через fit; C, alpha, n-граммы и пороги — настройки. C выбрано на validation, не по test. Чем меньше C, тем сильнее регуляризация.

Зачем Pipeline и отдельный test? Pipeline связывает fit TF-IDF и классификатора в одном процессе. Test проверяет перенос выбранного решения. Дубликаты и fit на всём корпусе способны завысить результат; постоянная настройка по test лишает его независимости.

Почему не обучали большую нейросеть? LR/NB — разумные CPU-базовые методы; T5 и fastText уже предобучены. Компактность не означает лучшее качество: T5 выбрана по ресурсам, её ограничения явно показаны.

Мы вычисляли MFCC и обнаруживали музыкальный плагиат? Нет. Мы суммировали статьи об этих методах. MFCC — спектральные аудиопризнаки, PCM — отсчёты звука, MIDI — музыкальные события. Сходство музыки само по себе не устанавливает нарушение права.

Почему не казахский у LR? В обучении выбраны 6 других языков с похожей парой Spanish/Portuguese. Казахский не был добавлен как класс. Нельзя считать неправильную русскую метку на казахском вводе успехом. FastText поддерживает более широкий набор языков.

Наш LR лучше fastText вообще? Нет. Это один test на шести сбалансированных языках, у fastText 176 возможных ответов и другое обучение. На коротком вводе результаты меняются. Возможное пересечение Wikipedia-источников дополнительно ограничивает сравнение.

Почему ручная аннотация не доказывает качество T5? Это отдельная авторская редактура. Метрики автоматического кандидата рассчитаны на настоящем выводе generate. Подмена этих результатов дала бы неверное описание эксперимента.

Технические детали, которые достаточно понимать

`fetch()` кеширует загрузки; SHA-256 проверяет совпадение с ожидаемым файлом, не его языковое качество. У моделей есть ожидаемые суммы; у CSV фактические хеши записываются. `run_manifest.json` сохраняет версии, источники и размеры. Он не сохраняет обученные объекты в памяти: после перезапуска ядра нужен Run all. Seed повышает повторяемость, но не гарантирует одинаковые вычисления на любом железе.

Логика выступления

Для каждой задачи: **какие данные → какой метод → как проверили → что получилось → какое ограничение**. Показывать: split и macro F1 отзывов; настоящий вывод T5 и пропуски; 4×10 песен и нормированные категории; 8+1 языковых моделей и график короткого ввода.

Главные числа: **76.87% / 0.7057** у sentiment LR; **35 слов и неполнота** у T5; **40 песен, 15 общих редких слов**; **99.06% на абзацах и 84.51% на трёх словах** у основной языковой LR. Не заявлять, что один процент доказывает понимание любого текста.